

Домашнее задание. OpenAPI-спецификация REST API интернет-магазина

1. Контекст

Вы — backend-разработчик интернет-магазина. Продуктовая команда приняла решение, что сначала фиксируется **контракт API**, и только потом пишется код. Ваш артефакт — не код, а **машиночитаемая спецификация OpenAPI**: по ней разработчик фронтенда/интеграций сможет построить клиент, а кодогенератор — сгенерировать SDK, и при этом у них не должно остаться вопросов, которые придётся задавать вам.

В систему входят доменные объекты: **каталоги, категории, товары, заказы**. Спецификация должна полностью их покрывать.

2. Что сдать

№	Файл	Описание
1	shop-api.openapi.yaml	Спецификация, YAML, OpenAPI 3.0.3 или 3.1.0
2	validation.txt	Вывод линтера (п. 6) без ошибок: 0 errors
3	postman_collection.json (опционально)	Коллекция, сгенерированная из вашей спеки (бонус)

Срок: **7 дней** с даты выдачи.

Формат сдачи: архив или репозиторий по ссылке.

Объём спеки: ориентировочно 1000–2500 строк YAML — меньше, чем нужно, это сигнал, что требования не выполнены; существенно больше — сигнал, что вы дублируете схемы.

3. Правила

1. Спецификация должна **валидироваться** без ошибок (п. 7) и рендериться в ReDoc без «дыр» (описания, примеры, ссылки на схемы на месте).
2. Любое место в спеке вы должны уметь объяснить при защите. «Так выдал генератор» — не является ответом.
3. Копирование чужих готовых спеки/ответов — оценка 0. Использование тулкита (Stoplight, Swagger Editor, Redocly, Spectral) — разрешено и поощряется.
4. Все идентификаторы, enum и форматы ниже — **обязательные**. Отклонения (например, цена в number (float) или с копейками) снимают баллы по п. 7.

4. Спецификация

4.1. Доменная модель

Сущности и связи

```
Catalog 1 — N Category
Category 1 — N Product
Product N — 1 Category
Order 1 — N OrderItem (вложена в Order)
OrderItem N — 1 Product (только product_id; цена фиксируется копией)
```

Пояснение про OrderItem: в заказе **не живёт ссылка на текущую цену товара**, а хранится unit_price — копия цены на момент оформления. Если цена товара изменится после оформления, история заказов не искажается.

Словари типов

Тип	OpenAPI	Комментарий
Идентификаторы	type: integer, format: int64	кроме Product.id — string, format: uuid
Денежные суммы	type: integer, format: int64	целые рубли (без копеек, без number/float)
Дата/время	type: string, format: date-time	RFC 3339, UTC
URL	type: string, format: uri	
UUID	type: string, format: uuid	

4.2. Схемы (components/schemas)

Все схемы — только в components/schemas, в операциях — только \$ref. У каждого поля: type + ограничения + description. required — явно. API работает только в рублях (RUB): цены и суммы — целые рубли, поля currency нет.

Catalog (каталог — верхний уровень иерархии, например «Электроника», «Одежда»)

Поле	Тип	Required	Ограничения
id	int64	да (в ответе)	генерируется сервером
name	string	да	3–100 символов
slug	string	да	2–60, $^{\wedge}[a-z0-9]+(-[a-z0-9]+)*$, уникален , только в запросе
description	string	нет	0–500
is_active	boolean	нет	default true
created_at, update	date-time	да (в ответе)	только в ответе, в запросах игнорируются — так и напишите в описании

Category (категория; иерархия: «Электроника → Ноутбуки → Игровые ноутбуки»)

Поле	Тип	Required	Ограничения
id	int64	да (в ответе)	
catalog_id	int64	да	каталог должен существовать, иначе 422
name	string	да	2-120
slug	string	да	^[a-z0-9]+(-[a-z0-9]+)*\$, уникален внутри каталога
image_url	uri	нет	
sort_order	int32	нет	default 0
is_active	boolean	нет	default true
created_at, update	date-time	да (в ответе)	

Product (товар)

Поле	Тип	Required	Ограничения
id	uuid	да (в ответе)	генерируется сервером (v4)
sku	string	да	5-32, ^[A-Z0-9-]+\$ уникален , нарушение — 409
name	string	да	2-200
description	string	нет	0-5000
category_id	int64	да	категория существует и is_active: true, иначе 422
price	int64	да	рубли, 0 ... 10 000 000
stock	int32	да	0 ... 1 000 000
status	enum	да	draft active archived, default draft
images	array of uri	нет	0-10
created_at, update	date-time	да (в ответе)	

Схемы запроса/ответа разделите: Product (полная, для ответа), ProductCreate (request body для POST), ProductUpdate (для PATCH, JSON Merge Patch, RFC 7386 — все поля optional). Так вы покажете, что понимаете разницу контрактов на чтение и запись.

Order (заказ)

Поле	Тип	Required	Ограничения
id	int64	да (в ответе)	
number	string	да (в ответе)	генерируется сервером: ^ORD-\d{4}-\d{6}\$ (например ORD-2026-000042), уникален
customer	объект Customer	да	см. ниже
items	array of OrderItem	да	1–100 элементов
status	enum	да	pending confirmed shipped delivered cancelled, default pending (в запросе создания — не передается)
total_amount	int64	да (в ответе)	сумма unit_price × quantity по items, в рублях, считается сервером
created_at, update	date-time	да (в ответе)	

Customer

Поле	Тип	Required	Ограничения
name	string	да	2–120
email	string, format email	да	
phone	string	да	E.164: ^\+[1-9]\d{6,14}\$

OrderItem

Поле	Тип	Required	Ограничения
product_id	uuid	да	товар существует, status: active, stock > 0, иначе 422
quantity	int32	да	1–999
unit_price	int64	да (в ответе)	не передается клиентом — сервер копирует Product.price на момент создания

			заказа. В схеме запроса этого поля нет — это принципиально
--	--	--	--

4.3. Эндпоинты

Все пути — с префиксом версии /v1 (пример):

№	Метод	URI	Назначение	Успех	Scope (security)
1	GET	/v1/catalogs	список катало (пагинация)	200	публичный

Дополнительно к эндпоинту 9 — параметры:

Параметр	Тип	Описание
q	string	поиск по name/sku, ≥ 2 символа
category_id	int64	
status	enum	
min_price / max_price	int64	рубли
sort	enum	name price created_at, default created_at
direction	enum	asc desc, default desc

Матрица статусов заказа (обязательно опишите в description операции № 18; несуществующий переход — **422**):

```

pending → confirmed → shipped → delivered
  ↓           ↓           ↓           ↓
cancelled (конец)  (конец)  (конец)

```

Финальные: delivered, cancelled.

4.4. Общие правила ответов

1. POST при успехе: **201** + заголовок Location: /v1/<resource>/<id> + тело объекта.

2. DELETE при успехе: **204**, без тела, без Content-Type.
3. Медиа-тип успеха: application/json; charset=utf-8.
4. Медиа-тип ошибок: **`application/problem+json`** (RFC 9457) — для всех 4xx/5xx во всех операциях.

4.5. Пагинация (фиксированный формат)

Применяется ко всем операциям-спискам (№ 1, 6, 8, 9, 16).

Запрос (query-параметры):

Параметр	Тип	Default	Ограничения
page	int32	1	min 1
limit	int32	20	1-100

Ответ — **конверт** (один и тот же components/schemas/Pagination для всех списков):

```
{
  "data": [ { "id": 1 } ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total_items": 137,
    "total_pages": 7,
    "has_next": true,
    "has_prev": false
  }
}
```

data — массив соответствующего типа. Конверт опишите схемой (можно через oneOf/anyOf на список или отдельными схемами CatalogList, ProductList — на ваше усмотрение, но дублировать pagination в каждой не можно).

4.6. Ошибки (фиксированный формат)

Тело каждой ошибки:

```
{
  "type": "https://api.shop.example/errors/product-not-found",
  "title": "Product not found",
  "status": 404,
  "detail": "Product with id '3fa85f64-5717-4562-b3fc-2c963f66afa6' was not found",
  "instance": "/v1/products/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "errors": [
    { "field": "sku", "code": "duplicate", "message": "SKU 'ABC-123' already exists" }
  ]
}
```

- `type` — URI вида `https://api.shop.example/errors/<kebab-case>`; задайте для каждой используемой ошибки свой `type` и перечислите их в `info.description` (таблица типов).
- `errors[]` обязателен для **422** (поле, код, человекочитаемое сообщение); для остальных кодов — опционален.
- `title` — человекочитаемое, `detail` — контекст (что именно не так).

Обязательный набор кодов и когда они используются:

Код	Когда	Пример
400	кривой запрос (не-JSON, битый uuid в пути)	productId не является UUID
401	нет токена / токен просрочен	
403	токен есть, но нет нужного scope	GET /v1/orders без orders:read
404	ресурс не существует	GET /v1/products/<uuid>
409	конфликт состояния/уникальности	дубликат sku; удаление каталога с категориями; idempotency-key с другим телом
422	тело валидно по схеме, но нарушает бизнес-правило	товар stock: 0 в заказе; товар не active; недопустимый переход статуса
429	превышен лимит (100 req/min на токен)	обязателен заголовок Retry-After (секунды)

500	непредвиденная ошибка	type: .../internal-server-error, без деталей инсайда
-----	--------------------------	--

Плюс в 429 опишите заголовки X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset (в components/headers).

4.7. Идемпотентность (фиксированное поведение)

- POST /v1/orders (и опционально POST /v1/products) принимает заголовков **`Idempotency-Key`** (uuid).
- Повторный запрос с тем же ключом **и тем же телом** за 24 часа → вернуть сохранённый результат первого (тот же id, тот же number), код 201.
- Тот же ключ, **другое тело** → **409**, type: .../idempotency-key-reuse.
- Без ключа — запрос принимается, но гарантии нет (напишите это в description).
- Ключ объявите в components/parameters и подвяжите к операциям.

4.8. Обязательные атрибуты спецификации (чек-лист)

- ☐ openapi: 3.0.3 (или 3.1.0) — один и тот же стиль по всему документу
- ☐ info: title, version: 1.0.0, contact, description (markdown, ≥ 10 строк: назначение API, таблица типов ошибок, политика версионирования, идемпотентность, rate limit)
- ☐ servers: минимум два — staging и production, с description
- ☐ tags: catalogs, categories, products, orders — с description; у каждой операции — operationId (camelCase), summary (≤ 60

символов), `description` (развёрнуто: что делает, коды ошибок, примечания)

- ☐ все схемы в `components/schemas`, все переиспользуемые параметры/ответы/ заголовки в `components/`
- ☐ у **каждого** параметра, запроса и ответа — `example` или `examples` (реалистичные данные: реальные цены в рублях, E.164 телефоны, UUID)
- ☐ пути параметров — snake/camel-единообразно, `catalogId` vs `catalog_id` — выберите одно и держите (уточните, что выбрали, в `info.description`)

4.9. Версионирование

URI-версионирование: текущая версия — `/v1`. В `info.description` опишите политику:

- аддитивные изменения (новые поля, новые эндпоинты) — **без** смены версии;
- ломающие (переименование полей, смена типов, удаление) — ``/v2``, старая версия продолжает жить ≥ 6 месяцев с заголовком `Deprecation: true` и `Sunset: <RFC 1123 date>` в ответах.

5. Защита: примерный список вопросов

Письменные ответы не сдаются. На защите вы показываете спеку (ReDoc) и отвечаете на вопросы. Ниже — примерный список: готовьте ответы **на своей спеке**, ссылаясь на конкретные `operationId`, схемы и коды состояния.

1. Какие из ваших методов идемпотентны и почему? Какие — нет?

2. Что произойдёт, если клиент дважды отправит один и тот же POST /v1/orders — без Idempotency-Key и с ним? Как это предотвращается на стороне сервера?
3. Как в вашей спецификации реализована пагинация (параметры, конверт ответа)? В чём её слабое место на больших данных?
4. В чём разница 400 / 401 / 403 / 409 / 422? Для каждого кода — пример из вашей спецификации.
5. Почему версионирование через URI (/v1), а не через заголовки? Как клиенты узнают, что вышла новая версия?
6. Как клиенты получают актуальный контракт API? Почему спецификация — «источник истины», а не вики-дока?
7. В каких случаях этот магазин лучше вести на GraphQL, в каких — на gRPC? Почему здесь выбран REST?
8. Почему у товара и PUT, и PATCH, а у каталога только PATCH (либо обоснуйте свой выбор методов обновления)?

6. Контроль качества спецификации

Перед сдачей прогнать (один из инструментов, вывод — в `validation.txt`):

```
# вариант 1 (Redocly)
npx @redocly/cli lint shop-api.openapi.yaml

# вариант 2 (Spectral, правила OAS)
npx @stoplight/spectral-cli lint shop-api.openapi.yaml

# вариант 3 (Python)
pip install openapi-spec-validator
openapi-spec-validator shop-api.openapi.yaml
```

Критерий: **0 errors**. Warnings допустимы, но каждый warning в `validation.txt` прокомментируйте: почему оставляете.

Прогоните рендер: `npx @redocly/cli preview-docs shop-api.openapi.yaml` — проверьте глазами: все схемы открываются, примеры отображаются, таблицы в `description` отрисовываются.

7. Критерии оценивания (30 баллов + 5 бонус)

№	Критерий	Баллы
1	Валидность: линтер 0 errors, ReDoc рендерится	3
2	Структура: info (полный description), servers, tags, operationId, единый стиль	3
3	Схемы: все 6 сущностей, все поля с ограничениями, типы по словарю, \$ref без дублей, разделение Create/Update/Read-схем	6
4	Эндпоинты: все 18, правильные методы/коды, Location/204, матрица статусов, фильтры /v1/products	6
5	Пагинация: конверт, единый Pagination, применён ко всем спискам	3
6	Ошибки: problem+json, все 8 кодов, таблица типов, заголовки 429, errors[] для 422	3
7	Безопасность и идиоматичность: scopes точно, Idempotency-Key со всей семантикой	3
8	Примеры и описания: у каждой операции/схемы, реалистичные данные	1,5
9	Защита: ответы на вопросы из § 5, привязанные к вашей спецификации	1,5
10	Бонус: Postman-коллекция из спецификации / CI-линтер / вопрос 7	+5

8. Стартовый каркас (пример формата, НЕ решение)

Скаффолд (основа), показывающий ожидаемую структуру. Заполнять полностью вам:

```
openapi: 3.0.3
info:
  title: Shop API
  version: 1.0.0
  description: |
    ## Назначение
```

REST API интернет-магазина: каталоги, категории, товары, заказы.

```
## Ошибки
Все ошибки – `application/problem+json` (RFC 9457). Таблица типов:
... (дополнить)
contact:
  name: API Team
  email: api@shop.example
servers:
  - url: https://staging.api.shop.example/v1
    description: Staging
  - url: https://api.shop.example/v1
    description: Production
tags:
  - name: catalogs
    description: Управление каталогами
  - name: categories
    description: Иерархия категорий внутри каталогов
  - name: products
    description: Карточки товаров
  - name: orders
    description: Оформление и обработка заказов
security:
  - bearerAuth: []
paths:
  /products:
    get:
      tags: [products]
      operationId: listProducts
      summary: Список товаров
      description: Пагинация + фильтры (q, category_id, status, min_price, max_price, sort, direction).
      security: []
      parameters:
        - $ref: '#/components/parameters/page'
        - $ref: '#/components/parameters/limit'
        # ... остальные параметры
      responses:
        '200':
          description: Список товаров
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ProductList'
              example:
                data:
                  - id: 3fa85f64-5717-4562-b3fc-2c963f66afa6
                    sku: NB-IGR-156
                    name: Игровой ноутбук 15.6"
                    price: 14999
                    status: active
                    stock: 12
                pagination:
                  page: 1
                  limit: 20
                  total_items: 1
                  total_pages: 1
                  has_next: false
                  has_prev: false
        '429':
          $ref: '#/components/responses/TooManyRequests'
        '500':
          $ref: '#/components/responses/InternalServerError'
components:
  securitySchemes:
    bearerAuth:
```

```
    type: http
    scheme: bearer
    bearerFormat: JWT
    description: JWT-токен. Scopes: catalogs:read, catalogs:write, products:read, products:write,
orders:read, orders:write
  parameters:
    page:
      name: page
      in: query
      description: Номер страницы
      required: false
      schema:
        type: integer
        format: int32
        minimum: 1
        default: 1
    limit:
      name: limit
      in: query
      description: Размер страницы
      required: false
      schema:
        type: integer
        format: int32
        minimum: 1
        maximum: 100
        default: 20
  responses:
    NotFound:
      description: Ресурс не найден
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/Error'
          example:
            type: https://api.shop.example/errors/not-found
            title: Not found
            status: 404
            detail: Resource was not found
            instance: /v1/products/3fa85f64-5717-4562-b3fc-2c963f66afa6
  schemas:
    Error:
      type: object
      required: [type, title, status, detail, instance]
      properties:
        type:
          type: string
          format: uri
          description: URI типа ошибки
        title:
          type: string
          description: Краткое человекочитаемое описание
        status:
          type: integer
          description: HTTP-код
        detail:
          type: string
          description: Контекст: что именно не так
        instance:
          type: string
          description: URI запроса, вызвавшего ошибку
        errors:
          type: array
          description: Полевые ошибки (обязателен при 422)
          items:
```

```
type: object
required: [field, code, message]
properties:
  field:
    type: string
  code:
    type: string
  message:
    type: string
```

Защита: до 10 минут (в зависимости от количества выполненных работ) — 5 минут показываете спеку в ReDoc, 5 минут отвечаете на вопросы (примерный список — § 5). Если вы не понимаете, зачем то или иное поле в вашей же спеке, — это минус 5 баллов за вне зависимости от валидности.